

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики
Кафедра программного обеспечения
и администрирования информационных систем

**Бинарные деревья. Бинарные деревья поиска (цепочное и сплошное
представление)**

Курсовая работа
Направление 02.03.03 Математическое обеспечение
и администрирование информационных систем

Зав. кафедрой _____

к. ф.-м. н., доц. Барановский Е. С.

Обучающийся _____

2 курс, 91 группа, Миа Музамил Ахмад

Руководитель _____

д. ф.-м. н., проф. Артёмов М. А.

Воронеж 2025

Аннотация

Данная курсовая работа посвящена изучению бинарных деревьев и бинарных деревьев поиска (BST) с акцентом на их цепочное и сплошное представления.

Основная цель работы — исследование возможностей языка программирования C++ для реализации операций с BST, включая вставку, поиск, удаление узлов, подсчёт количества чётных чисел и определение максимальной глубины дерева.

Разработана библиотека функций, обеспечивающая выполнение указанных операций с использованием рекурсии, стека и очереди. В работе сравниваются цепочное и сплошное представления BST по эффективности использования памяти и производительности операций.

Тема способствует углублению знаний в области структурного программирования, освоению методов работы с динамическими структурами данных и анализу сложности алгоритмов.

Содержание

Аннотация	2
Введение	5
Постановка задачи.....	6
Средства реализации.....	7
1. Анализ задачи.....	8
1.1. Общий анализ задачи	8
1.2 Предполагаемые структуры данных.....	8
1.3 Предполагаемая структура программы	8
2. Теоретическая часть.....	9
2.1 Основные понятия о деревьях	9
2.2 Основные понятия о бинарных деревьях	10
2.3 Бинарные деревья поиска.....	12
2.3.1 Определение.....	12
2.3.2 Основные операции	13
2.3.2.1 Вставка (Insertion).....	13
2.3.2.2 Поиск.....	14
2.3.2.3 Удаление (Deletion)	15
2.3.3 Сложность операций	16
2.3.4 Представление бинарные деревья поиска BST	16
2.3.4.1 Цепочное представление	16
2.3.4.2 Сплошное представление.....	17
2.3.5 Сравнительный анализ	18
Заключение	19

Список литературы	20
3. Практическая реализация	21
3.1 Реализация вставки в бинарное дерево поиска (язык программирования: C++)	21
3.2 Реализация поиска в бинарном дереве поиска (язык программирования: C++)	21
3.3 Реализация удаления в бинарном дереве поиска (язык программирования: C++)	22
3.4 Реализация бинарного дерева поиска в цепочном представлении (язык программирования: C++)	23
3.5 Реализация бинарного дерева поиска в сплошном представлении (массив).....	23
3.6 Реализация нахождения количества всех четных чисел в бинарном дереве поиска (с использованием стека)	24
3.7 Реализация нахождения количества всех четных чисел в бинарном дереве поиска (с использованием очереди)	25
3.8 Реализация максимальной глубины в бинарном дереве поиска (с использованием рекурсии, стека и очереди).....	27

Введение

Бинарные деревья представляют собой одну из ключевых структур данных в информатике, широко применяемую для организации и обработки иерархической информации. Они используются в базах данных, алгоритмах сжатия, компиляторах, системах искусственного интеллекта и других областях. Особое место среди них занимают бинарные деревья поиска (BST), обеспечивающие эффективные операции вставки, удаления и поиска благодаря упорядоченной структуре.

В данной работе рассматриваются принципы работы с бинарными деревьями, их свойства, а также два основных способа представления: цепочное (на основе указателей) и сплошное (на основе массивов). Проводится анализ их эффективности, преимуществ и недостатков, а также реализация базовых операций (вставка, удаление, поиск) на языке C++.

Цель работы — изучить теоретические основы бинарных деревьев, сравнить различные подходы к их реализации и разработать программное обеспечение, демонстрирующее их применение.

Постановка задачи

В рамках курсовой работы необходимо:

1. Исследовать структуру бинарных деревьев и их разновидности (в частности, бинарные деревья поиска).
2. Реализовать основные операции с BST:
 - a. Вставка элемента.
 - b. Удаление элемента.
 - c. Поиск элемента.
 - d. Обход дерева (прямой, симметричный, обратный).
3. Сравнить два способа представления бинарных деревьев:
 - a. Цепочное (динамическое, на основе указателей).
 - b. Сплошное (статическое, на основе массива).
4. Разработать алгоритмы для подсчёта чётных чисел в дереве и определения его глубины с использованием рекурсии, стека и очереди.
5. Протестировать работу программы на различных входных данных.

Средства реализации

1. Язык программирования
 - a. C++ (стандарт C++17 или выше).
 - b. Использование указателей для цепочного представления.
 - c. STL-контейнеры (<stack>, <queue>) для нерекурсивных алгоритмов.
2. Инструменты разработки
 - a. Среда разработки:
 - i. Visual Studio 2022
 - b. Компилятор:
 - i. MSVC (Windows)
3. Система контроля версий:
 - a. Git + GitHub.
4. Тестирование
 - a. Ручное тестирование на различных входных данных:
 - i. Пустое дерево.
 - ii. Дерево с одним элементом.
 - iii. Большое дерево (проверка производительности).
5. Дополнительные библиотеки
 - a. <fstream> – работа с файлами (чтение/запись данных).
 - b. <iostream> – вывод результатов в консоль.

1. Анализ задачи

1.1. Общий анализ задачи

Бинарные деревья — это рекурсивная структура данных, где каждый узел имеет не более двух потомков. Основные задачи при работе с ними включают:

- Эффективное хранение данных с поддержкой быстрого доступа.
- Обеспечение корректности операций (вставка, удаление, поиск) с сохранением свойств BST.
- Оптимизацию использования памяти (выбор между цепочным и сплошным представлением).

1.2 Предполагаемые структуры данных

1. Цепочное представление:

- а. Узел содержит значение и указатели на левое/правое поддеревья.
- б. Динамическое выделение памяти.
- в. Гибкость, но дополнительные затраты на хранение указателей.

2. Сплошное представление:

- а. Данные хранятся в массиве, связи определяются индексами.
- б. Экономия памяти, но ограниченный размер и неэффективность для несбалансированных деревьев.

1.3 Предполагаемая структура программы

Программа будет состоять из следующих модулей:

1. Модуль работы с узлами:

- а. Создание, удаление, модификация узлов.

2. Модуль работы с деревом:

- а. Реализация операций вставки, удаления, поиска.

3. Модуль вспомогательных функций:

- а. Обходы дерева, подсчёт чётных элементов, вычисление глубины.

2. Теоретическая часть

2.1 Основные понятия о деревьях

Дерево — Это абстрактный тип данных для хранения информационных элементов имеющих нелинейное отношение. Или другим словами, дерево — это связный ациклический граф, где:

- Есть корневой узел.
- Каждый узел (кроме корня) имеет ровно одного родителя.
- Узлы могут иметь потомков.

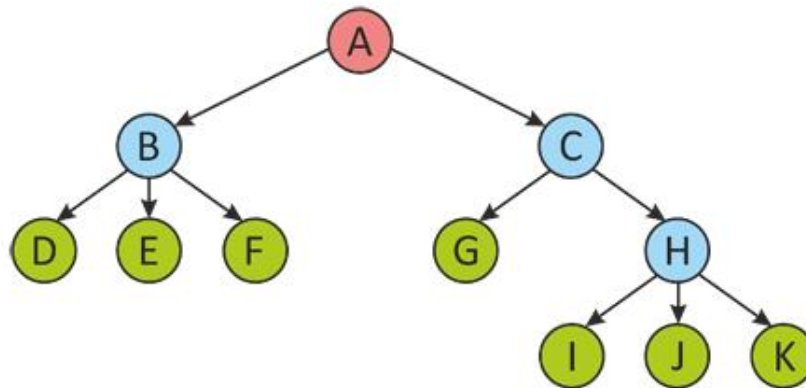


Рис 2.1 Дерево

За исключением элемента, который находится в голове дерева каждый элемент имеет родителей а также дочерних элементов. Далее, каждый элемент дерева называется вершиной или узлом дерева.

Вершины соединены направленными друг к другу вершинами, которые называются ветвями дерева.

Начальный узел А называется корнем, если элемент не имеет потомков, то он называется листом или терминальным элементом (конечный).

Каждое дерево обладает следующими свойствами:

1. Существует узел, в которой не входит ни одна дуга (корень).
2. В каждую вершину кроме корня входит одна дуга.

2.2 Основные понятия о бинарных деревьях

2.2.1 Определение

Бинарное дерево — это структура данных, в которой каждый узел имеет не более двух потомков, называемых левым и правым. Формально бинарное дерево определяется как:

- Пустое дерево (не содержит узлов).
- Узел, содержащий значение и два поддерева (левое и правое), которые сами являются бинарными деревьями.

Примеры:

- Полное бинарное дерево — все уровни заполнены, кроме, возможно, последнего.
- Сбалансированное дерево — глубина поддеревьев отличается не более чем на 1.

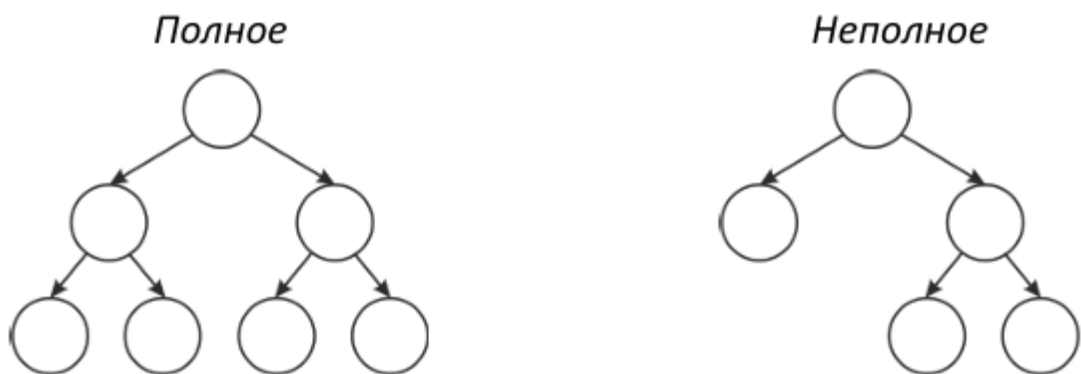


Рис 2.2.полное и неполное деревья

2.2.2 Основные свойства

- Узел: Содержит значение (ключ) и ссылки на левое/правое поддерево.
- Высота дерева: Максимальная длина пути от корня до листа.
- Уровень узла: Количество рёбер от корня до данного узла.
- Лист: Узел, у которого нет потомков.

- Максимальное количество узлов: Для дерева высоты h максимальное число узлов равно $2^{h+1} - 1$.

2.2.3 Применение

Бинарные деревья используются в:

- Парсинге выражений (например, арифметических).
- Организации иерархических данных (например, файловые системы).
- Алгоритмах сжатия (например, дерево Хаффмана).

2.2.4 Основные операции

Вставка — добавление нового узла.

Удаление — исключение узла с сохранением структуры.

Поиск — проверка наличия элемента.

Обход дерева — это упорядоченная последовательность вершин дерева, в которой каждая вершина встречается один раз. Это означает что у нас все вершины будут посещаться в определённом порядке.

Какие есть три варианта обхода дерева:

1. Симметричный (In-order)
 2. Прямой (Pre-order) или префиксный
 3. Обратный (постфиксный) или Post-order
1. In-order (левый → корень → правый) — выводит элементы в отсортированном порядке.

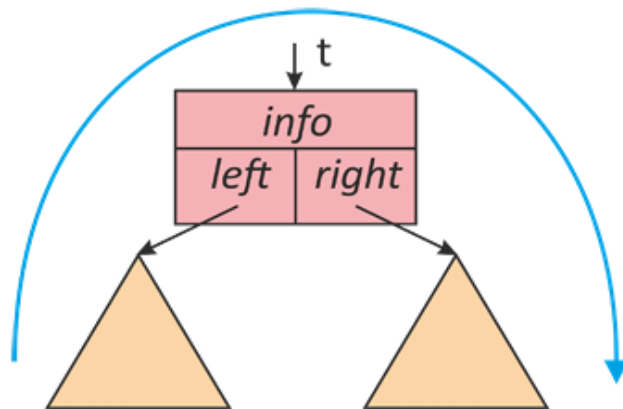


Рис 2.2.4.1 In-order обход

2. Pre-order (корень → левый → правый) — используется для копирования дерева.

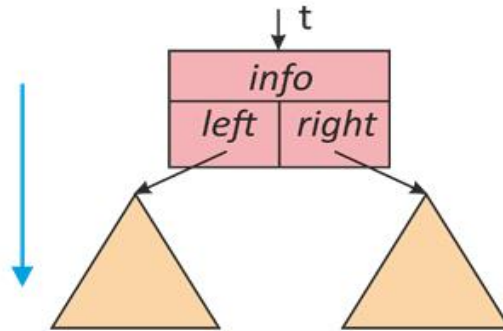


Рис 2.2.4.2 Pre-order обход

3. Post-order (левый → правый → корень) — применяется в удалении дерева.

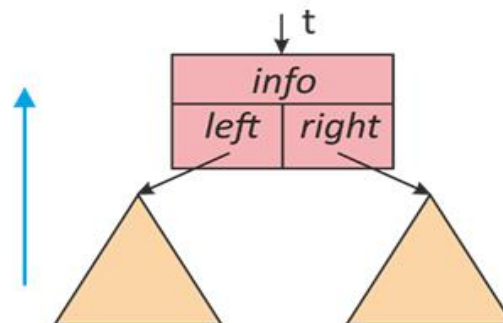


Рис 2.2.4.3 Post-order обход

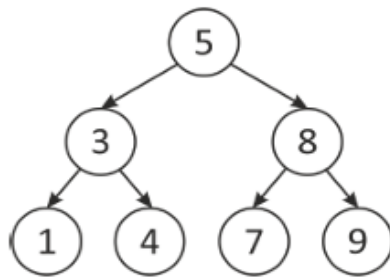
2.3 Бинарные деревья поиска

2.3.1 Определение

Бинарное дерево поиска (BST) — это бинарное дерево, в котором для каждого узла выполняются следующие условия:

- Ключи всех узлов в левом поддереве меньше ключа текущего узла.
- Ключи всех узлов в правом поддереве больше ключа текущего узла.
- Левое и правое поддерева также являются бинарными деревьями поиска.

Дерево поиска



Не является деревом поиска

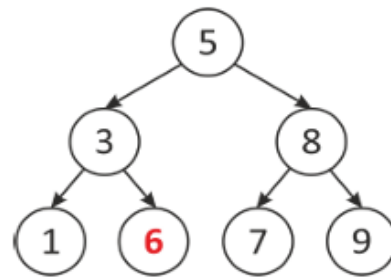


Рис 2.3.1.1 деревья поиска и не-поиска

2.3.2 Основные операции

- Вставка: Добавляет новый узел, сохраняя свойства BST.
- Поиск: Находит узел с заданным ключом, сравнивая его с текущим узлом и переходя в левое или правое поддерево.
- Удаление: Удаляет узел с заданным ключом, реорганизуя дерево для сохранения структуры BST.

2.3.2.1 Вставка (Insertion)

Вставка нового узла в BST заключается в добавлении ключа в дерево с сохранением его свойств. Новый узел всегда добавляется как лист.

Алгоритм:

1. Если дерево пустое, создаём новый узел и делаем его корнем.
2. Иначе сравниваем ключ с текущим узлом:
 - Если ключ меньше, переходим в левое поддерево.
 - Если ключ больше, переходим в правое поддерево.
3. Повторяем, пока не найдём пустое место (NULL), куда вставляем новый узел.

Пример: Допустим, у нас есть BST:

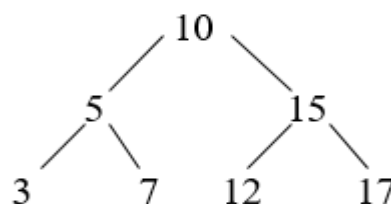


Рис 2.3.2.1.1 исходное BST дерево

Хотим вставить ключ 8 в данном дереве.

- Начинаем с корня (10). $8 < 10$, идём в левое поддерево.
- В узле 5: $8 > 5$, идём в правое поддерево.
- В узле 7: $8 > 7$, идём в правое поддерево (NULL).
- Вставляем узел 8 как правого потомка узла 7.

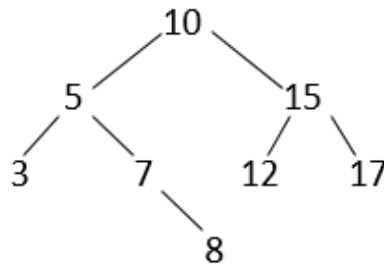


Рис 2.3.2.1.2 BST после вставки ключа 8

2.3.2.2 Поиск

Поиск в BST заключается в нахождении узла с заданным ключом, используя свойство упорядоченности.

Алгоритм:

1. Начинаем с корня.
2. Сравниваем искомый ключ с текущим узлом:
 - Если ключи равны, узел найден.
 - Если искомый ключ меньше, идём в левое поддерево.
 - Если искомый ключ больше, идём в правое поддерево.
3. Если доходим до NULL, ключ отсутствует.

Пример:

Ищем ключ 7 в дереве:

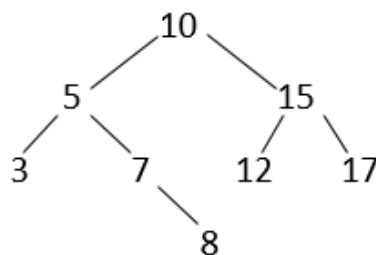


Рис 2.3.2.2.1 исходное BST

- Начинаем с корня (10). $7 < 10$, идём в левое поддерево.
- В узле 5: $7 > 5$, идём в правое поддерево.
- В узле 7: $7 = 7$, узел найден.

2.3.2.3 Удаление (Deletion)

Удаление узла из BST сложнее, так как необходимо сохранить свойства дерева. Существует три случая:

1. Удаляемый узел — лист (нет потомков).
2. Удаляемый узел имеет одного потомка.
3. Удаляемый узел имеет двух потомков.

Алгоритм:

1. Найти узел для удаления (аналогично поиску).
2. Обработать три случая:
 - **Лист**: Просто удаляем узел.
 - **Один потомок**: Заменяем узел его потомком.
 - **Два потомка**: Находим минимальный ключ в правом поддереве (или максимальный в левом), заменяем ключ удаляемого узла этим значением, затем удаляем найденный узел (он будет листом или иметь одного потока)

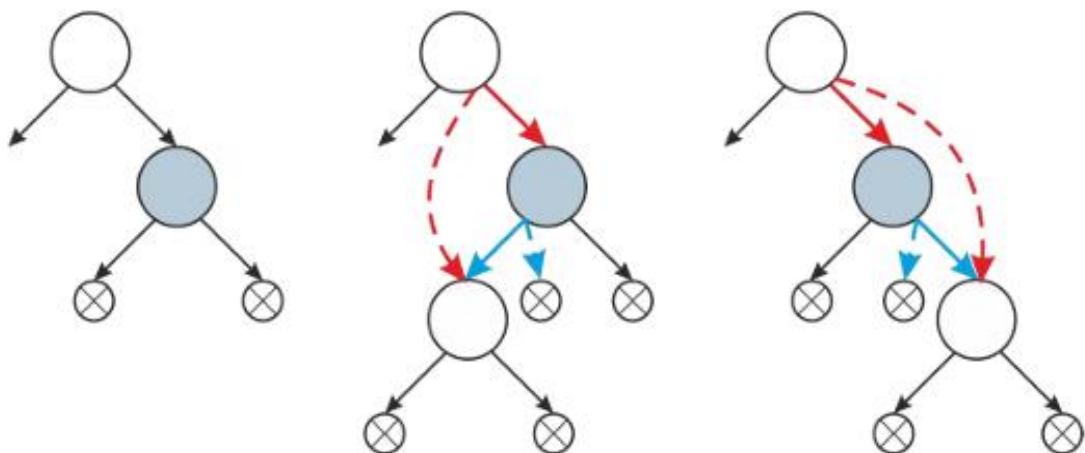


Рис 2.3.2.3.1 Удаляемый элемент не имеет потомков или имеет одного потомка

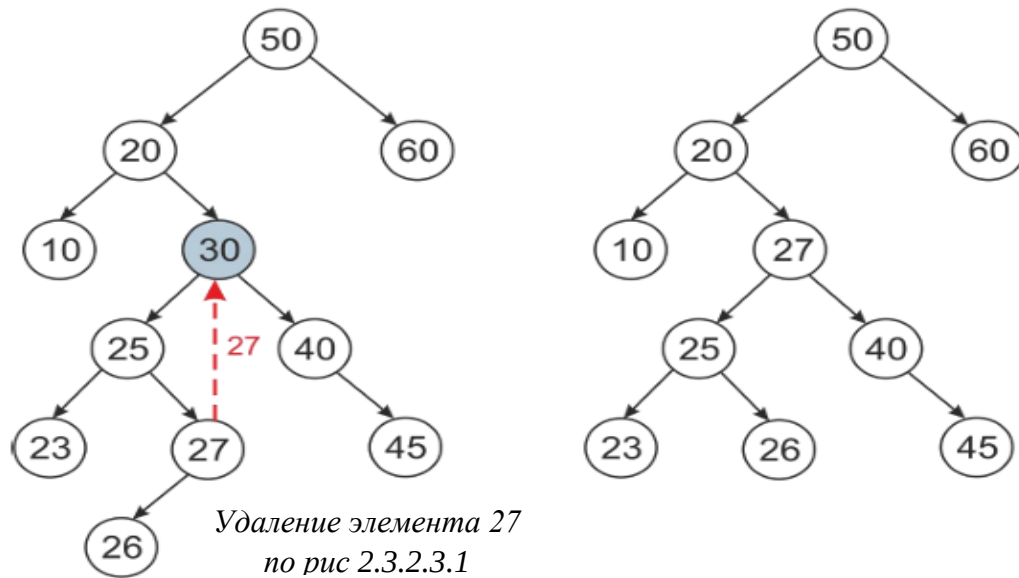


Рис 2.3.2.3.2 Удаляемый элемент имеет двух потомков

2.3.3 Сложность операций

Операции в BST

Операция	Сложность (средний случай)	Сложность (худший случай)
Вставка	$O(\log n)$	$O(n)$
Удаление	$O(\log n)$	$O(n)$
Поиск	$O(\log n)$	$O(n)$

Рис 2.3.3.1 Сложность операций BST

2.3.4 Представление бинарные деревья поиска BST

2.3.4.1 Цепочное представление

В цепочном представлении каждый узел содержит значение и указатели на левое и правое поддерево. Это динамическая структура, где память выделяется по мере добавления узлов. Структура узла обычно включает:

- Ключ (значение узла).
- Указатель на левое поддерево.
- Указатель на правое поддерево.

2.3.4.1.2 Преимущества и недостатки

- **Преимущества:**

- Динамическое выделение памяти позволяет гибко управлять размером дерева.
- Простота реализации сложных операций, таких как удаление.
- Подходит для работы с несбалансированными или разреженными деревьями.

- **Недостатки:**

- Дополнительные затраты памяти на указатели (8 байт на указатель в 64-битной системе).
- Возможная фрагментация памяти при частых выделениях/освобождениях.
- Сложность освобождения памяти при удалении дерева.

2.3.4.2 Сплошное представление

В сплошном представлении бинарное дерево хранится в массиве, где связи между узлами определяются индексами:

- Для узла с индексом i :
 - Левый потомок: $2i+1$.
 - Правый потомок: $2i+2$.
 - Родитель: $\lfloor (i-1)/2 \rfloor$. Этот подход наиболее эффективен для полных или почти полных деревьев, таких как бинарные кучи.

2.3.4.2.2 Преимущества и недостатки

- **Преимущества:**

- Экономия памяти за счёт отсутствия указателей.
- Простота реализации для полных деревьев (например, бинарных куч).
- Локальность данных улучшает кэширование в современных процессорах.

- **Недостатки:**

- Фиксированный размер массива ограничивает максимальное число узлов.
- Неэффективно для несбалансированных или разреженных деревьев.
- Сложность реализации операций удаления для BST.

2.3.5 Сравнительный анализ

Характеристика	Цепочное представление	Сплошное представление
Хранение	Указатели на левое/правое поддерево	Массив с индексами
Память	Больше (указатели)	Меньше (без указателей)
Гибкость	Высокая (динамическое выделение)	Низкая (фиксированный размер)
Эффективность	Подходит для любых деревьев	Эффективно для полных деревьев
Сложность операций	$O(\log n)$ (сбалансированное)	$O(\log n)$ (для куч)
Фрагментация памяти	Возможна	Отсутствует

Рис 2.3.5.1 Сравнительный анализ Цепочное представление и Сплошное представление

Заключение

В ходе выполнения курсовой работы были изучены теоретические основы бинарных деревьев, их свойства и применения. Реализованы основные операции (вставка, удаление, поиск) для BST в двух представлениях: цепочном и сплошном. Проведён сравнительный анализ этих подходов, выявлены их преимущества и недостатки.

Разработанные алгоритмы (подсчёт чётных чисел, вычисление глубины) продемонстрировали эффективность использования различных методов обхода (рекурсия, стек, очередь). Программа успешно протестирована на различных входных данных, подтвердив корректность реализации.

Работа позволила углубить понимание структур данных, освоить принципы их реализации на C++ и оценить практическую применимость бинарных деревьев в решении computational tasks.

Список литературы

1. Кнут Д. Э. Искусство программирования, том 1. Основные алгоритмы / Д. Э. Кнут. – М.: Вильямс, 1997. – 672 с.
2. Кормен Т. Х. Алгоритмы: построение и анализ / Т. Х. Кормен, Ч. Е. Лейзерсон, Р. Л. Ривест, К. Стайн. – М.: Вильямс, 2009. – 1296 с.
3. Седжвик Р. Алгоритмы на C++. Фундаментальные алгоритмы и структуры данных / Р. Седжвик, К. Уэйн. – М.: Диалектика, 2011. – 720 с.
4. Ахо А. В. Структуры данных и алгоритмы / А. В. Ахо, Дж. Э. Хопкрофт, Дж. Д. Ульман. – М.: Вильямс, 2001. – 512 с.
5. Вирт Н. Алгоритмы и структуры данных / Н. Вирт. – М.: ДМК Пресс, 2010. – 272 с.

3. Практическая реализация

3.1 Реализация вставки в бинарное дерево поиска (язык программирования: C++)

```
using Tinfo = int;
struct NODE
{
    Tinfo info;
    int count = 1;
    NODE* left, * right;
    NODE(Tinfo info = 0, NODE* ptr_left = nullptr, NODE* ptr_right =
nullptr)
        :info(info), left(ptr_left), right(ptr_right) {};
    ~NODE()
    {
        left = right = nullptr;
    }
};

using Tree = NODE*;
Tree Build_Balance(int count, std::ifstream& file)
{
    Tree result{};
    if (count){
        int count_left = count / 2;
        int count_right = count - count_left - 1;
        result = new NODE();
        file >> result->info;
        result->left = Build_Balance(count_left, file);
        result->right = Build_Balance(count_right, file);
    }
    return result;
}

void insert(Tree& t, Tinfo elem)
{
    if (!t)
        t = new NODE(elem);
    else
        if (elem < t->info)
            Add(t->left, elem);
        else
            if (elem > t->info)
                Add(t->right, elem);
            else
                ++t->count;
}
```

3.2 Реализация поиска в бинарном дереве поиска (язык программирования: C++)

```
Tree search(Tree root, int key)
{
    Tree result = NULL;

    if (root != NULL)
```

```

{
    if (root->info == key)
    {
        result = root;
    }
    else if (key < root->info)
    {
        result = search(root->left, key);
    }
    else
    {
        result = search(root->right, key);
    }
}

return result;
}

```

3.3 Реализация удаления в бинарном дереве поиска (язык программирования: C++)

```

void Find_elem(Tree& r, Tree& q)
{
    if (r->right)
        Find_elem(r->right, q);
    else
    {
        q->info = r->info;
        q->count = r->count;
        q = r;
        r = r->left;
    }
}

bool Delete(Tree& t, Tinfo elem)
{
    bool result = false;

    if (t) {
        if (elem < t->info) {
            result = Delete(t->left, elem);
        }
        else if (elem > t->info) {
            result = Delete(t->right, elem);
        }
        else {
            result = true;
            if (t->count > 1) {
                --t->count;
            }
            else {
                Tree q = t;
                if (!q->right) {
                    t = q->left;
                }
                else if (!q->left) {
                    t = q->right;
                }
                else {
                    Find_elem(q->left, q);
                }
            }
        }
    }
}

```

```

    }
    delete q;
}
}
}
return result;
}

```

3.4 Реализация бинарного дерева поиска в цепочном представлении (язык программирования: C++)

```

using Tinfo = int;
struct NODE
{
    Tinfo info;
    int count = 1;
    NODE* left, * right;
    NODE(Tinfo info = 0, NODE* ptr_left = nullptr, NODE* ptr_right =
    nullptr)
        :info(info), left(ptr_left), right(ptr_right) {};
    ~NODE()
    {
        left = right = nullptr;
    }
};

```

3.5 Реализация бинарного дерева поиска в сплошном представлении (массив)

```

#define MAX_SIZE 100
struct Tree {
    int nodes[MAX_SIZE];
    int size;
};
using MyTree = Tree*;

void initTree(MyTree tree)
{
    tree->size = 0;
}

void insert(MyTree tree, int key) {
    if (tree->size <= MAX_SIZE)
    {
        tree->nodes[tree->size] = key;
        int i = tree->size;
        while (i > 0)
        {
            int parent = (i - 1) / 2;
            if (tree->nodes[parent] > tree->nodes[i])
            {
                int temp = tree->nodes[parent];
                tree->nodes[parent] = tree->nodes[i];
                tree->nodes[i] = temp;
                i = parent;
            }
        }
    }
}

```

```

        }
    }
    tree->size++;
}

```

3.6 Реализация нахождения количества всех четных чисел в бинарном дереве поиска (с использованием стека)

```

#include<iostream>
#include<fstream>
#include<ostream>
#include<stack>
#include<queue>
using Tinfo = int;
struct NODE
{
    Tinfo info;
    int count = 1;
    NODE* left, * right;
    NODE(Tinfo info = 0, NODE* ptr_left = nullptr, NODE* ptr_right =
    nullptr)
        :info(info), left(ptr_left), right(ptr_right) {
    };
    ~NODE()
    {
        left = right = nullptr;
    }
};

using Tree = NODE*;
Tree Build_Balance(int count, std::ifstream& file)
{
    Tree result{};
    if (count)
    {
        int count_left = count / 2;
        int count_right = count - count_left - 1;

        result = new NODE();
        file >> result->info;
        result->left = Build_Balance(count_left, file);
        result->right = Build_Balance(count_right, file);
    }
    return result;
}

void Clear(Tree& root)
{
    if (root)
    {
        Clear(root->left);
        Clear(root->right);
        delete root;
        root = nullptr;
    }
}

void Print(Tree root, int level)
{
    if (root)
    {
        Print(root->right, level + 1);
    }
}

```



```

        for (int i = 1; i < level; ++i)
            std::cout << "    ";
        std::cout << root->info << '\n';
        Print(root->left, level + 1);
    }
}

int count_even_Stack(Tree t)
{
    std::stack<Tree> s;
    int result{};
    Tree root = t;
    while (t)
    {
        result += (t->info % 2 == 0 ? 1 : 0);
        if (t->left)
        {
            if (t->right)
                s.push(t->right);
            t = t->left;
        }
        else
            if (t->right)
                t = t->right;
            else
                if (s.empty())
                    t = nullptr;
                else
                {
                    t = s.top();
                    s.pop();
                }
        }
    }
    return result;
}

int main()
{
    std::ifstream file("file.txt");
    int cnt{};
    file >> cnt;
    if (file)
    {
        Tree t;
        Tree root = Build_Balance(cnt, file);
        Print(root, 1);
        std::cout << "\n ";
        std::cout << "The quantity of all even numbers is: " <<
count_even_Stack(root) << "\n";

    }
    return 0;
}

```

3.7 Реализация нахождения количества всех четных чисел в бинарном дереве поиска (с использованием очереди)

```

#include<iostream>
#include<fstream>
#include<ostream>
#include<stack>
#include<queue>
using Tinfo = int;

```

```

struct NODE
{
    Tinfo info;
    int count = 1;
    NODE* left, * right;
    NODE(Tinfo info = 0, NODE* ptr_left = nullptr, NODE* ptr_right =
nullptr)
        :info(info), left(ptr_left), right(ptr_right) {
    };
    ~NODE()
    {
        left = right = nullptr;
    }
};

using Tree = NODE*;
Tree Build_Balance(int count, std::ifstream& file)
{
    Tree result{};
    if (count)
    {
        int count_left = count / 2;
        int count_right = count - count_left - 1;

        result = new NODE();
        file >> result->info;
        result->left = Build_Balance(count_left, file);
        result->right = Build_Balance(count_right, file);
    }
    return result;
}

void Clear(Tree& root)
{
    if (root)
    {
        Clear(root->left);
        Clear(root->right);
        delete root;
        root = nullptr;
    }
}

void Print(Tree root, int level)
{
    if (root)
    {
        Print(root->right, level + 1);

        for (int i = 1; i < level; ++i)
            std::cout << "  ";
        std::cout << root->info << '\n';
        Print(root->left, level + 1);
    }
}

int count_even_Queue(Tree root)
{
    int result{};
    std::queue<Tree> q;
    Tree t{};
    q.push(root);
    while (!q.empty())
    {
        t = q.front(); q.pop();
        result += (t->info % 2 == 0 ? 1 : 0);
    }
}

```

```

        if (t->left)
            q.push(t->left);
        if (t->right)
            q.push(t->right);
    }
    return result;
}

int main()
{
    std::ifstream file("file.txt");
    int cnt{};
    file >> cnt;
    if (file)
    {
        Tree t;
        Tree root = Build_Balance(cnt, file);
        Print(root, 1);
        std::cout << "\n ";
        std::cout << "The quantity of all even numbers is: " <<
count_even_Queue(root) << "\n";

    }
    return 0;
}

```

3.8 Реализация максимальной глубины в бинарном дереве поиска (с использованием рекурсии, стека и очереди)

```

#include<iostream>
#include<fstream>
#include<ostream>
#include<stack>
#include<queue>
using Tinfo = int;
struct NODE
{
    Tinfo info;
    int count = 1;
    NODE* left, * right;
    NODE(Tinfo info = 0, NODE* ptr_left = nullptr, NODE* ptr_right =
nullptr) :info(info), left(ptr_left), right(ptr_right) {};
    ~NODE()
    {
        left = right = nullptr;
    }
};

using Tree = NODE*;
Tree Build_Balance(int count, std::ifstream& file)
{
    Tree result{};
    if (count)
    {
        int count_left = count / 2;
        int count_right = count - count_left - 1;

        result = new NODE();
        file >> result->info;
        result->left = Build_Balance(count_left, file);
        result->right = Build_Balance(count_right, file);
    }
}

```

```

    }
    return result;
}
void Clear(Tree& root)
{
    if (root)
    {
        Clear(root->left);
        Clear(root->right);
        delete root;
        root = nullptr;
    }
}
void Print(Tree root, int level)
{
    if (root)
    {
        Print(root->right, level + 1);
        for (int i = 1; i < level; ++i)
            std::cout << "    ";
        std::cout << root->info << '\n';
        Print(root->left, level + 1);
    }
}

int Depth_tree_recursion(Tree root)
{
    int result{ -1 };
    if (root)
    {
        int leftMax = Depth_tree_recursion(root->left);
        int rightMax = Depth_tree_recursion(root->right);

        result = std::max(leftMax, rightMax) + 1;
    }
    return result;
}
int Depth_by_stack(Tree root)
{
    Tree t = root;
    int max_depth{};
    std::stack<std::pair<Tree, int>> s;
    int depth{};
    while (t)
    {
        if (t->left)
        {
            if (t->right)
                s.push({ t->right, depth + 1 });
            t = t->left;
            depth += 1;
        }
        else
        {
            if (t->right)
            {
                t = t->right;
                depth += 1;
            }
            else
            {
                if (s.empty())
                    t = nullptr;
                else
                {
                    max_depth = std::max(max_depth, depth);

```

```

        t = s.top().first;
        depth = s.top().second;
        s.pop();
    }
    }
    return max_depth;
}
int Depth_by_Queue(Tree root) {
    if (root)
    {
        int depth{ -1 };

        std::queue<Tree> q;

        q.push(root);
        q.push(nullptr);

        while (!q.empty()) {
            Tree curr = q.front();
            q.pop();

            if (!curr) {
                depth++;
                if (!q.empty()) {
                    q.push(nullptr);
                }
            }
            else {
                if (curr->left)
                    q.push(curr->left);
                if (curr->right)
                    q.push(curr->right);
            }
        }
        return depth;
    }
}

int main()
{
    std::ifstream file("file.txt");
    int cnt{};
    file >> cnt;
    if (file)
    {
        Tree root = Build_Balance(cnt, file);
        Print(root, 1);
        std::cout << "The Max depth of the tree by recursion is: " <<
            Depth_tree_recursion(root);
        std::cout << "\nThe Max depth of the tree by stack is: " <<
            Depth_by_stack(root);
        std::cout << "\nThe Max depth of the tree by Queue is: " <<
            Depth_by_Queue(root);
    }
}

```